

浙江大学



基于 MindSpore 的 MNIST 手写体 数字识别实验 (CNN)

姓名：吴溪语

学号：3230103327

实验日期：2026 年 6 月 15 日

指导老师：李旻

摘 要

本次实验以 MNIST 手写数字识别为任务背景，在华为云 ModelArts Notebook 环境中使用 MindSpore 框架完成模型训练与测试。实验首先按照指导文档完成 MNIST 数据集读取、图像预处理、LeNet5 网络构建、模型训练和预测可视化。在此基础上，进一步对基础 LeNet5 结构进行了调整，并将改进模型与基础模型的测试结果进行对比。

改进模型主要增加了卷积层输出通道数，并调整了全连接层规模，以提高网络对手写数字笔画、轮廓和局部结构的特征提取能力。根据当前 notebook 中保存的运行结果，基础 LeNet5 第三轮训练后在测试集上的准确率为 98.6%，平均损失为 0.043426；改进 LeNet5 在第三轮训练后准确率达到 98.8%，平均损失下降到 0.037108。结果表明，基础 LeNet5 已经能够在 MNIST 数据集上取得较高准确率，改进结构带来的准确率提升较为有限，但平均损失有所下降，说明模型在测试集上的整体误差进一步降低。

关键词： MindSpore；LeNet5；卷积神经网络；MNIST；手写体数字识别

Contents

1	实验目的	1
2	实验任务	1
3	实验原理	1
3.1	MNIST 数据集与预处理	1
3.2	卷积神经网络与 LeNet5 结构	2
3.3	损失函数与评价指标	2
4	实验内容	3
4.1	实验平台与运行环境	3
4.2	MNIST 数据集加载	4
4.3	数据读取与规范化处理	4
4.4	基础 LeNet5 模型实现	5
4.5	改进 LeNet5 结构设计	6
4.6	模型训练与测试方法	7
4.7	预测结果可视化	8
5	实验结果	9
5.1	基础模型训练表现	9
5.2	改进模型训练表现	10
5.3	最终性能对比	10
6	结果分析	12
6.1	训练轮数与收敛状态	12
6.2	网络结构变化的影响	12
6.3	实验不足与改进方向	12
7	实验总结	13
A	核心代码	14

1 实验目的

本实验旨在基于 MindSpore 完成 MNIST 手写数字识别任务，并熟悉深度学习模型从数据处理到训练评估的基本流程。通过本实验，可以了解图像数据的读取方式，掌握尺寸调整、归一化和格式转换等预处理操作的作用。同时，进一步理解卷积神经网络如何从图像中提取特征、得到分类结果，以及损失函数和优化器如何参与模型参数更新。

在模型设计方面，实验首先使用经典 LeNet5 作为基础模型。该模型结构清晰，由卷积层、池化层和全连接层组成，适合处理 MNIST 这类单通道、小尺寸的灰度图像。在完成基础模型训练后，实验进一步设计改进模型，主要调整卷积层通道数和全连接层维度，并通过 Accuracy 和 Avg loss 两个指标比较两种模型的测试表现，从而分析网络结构调整对识别效果的影响。

2 实验任务

本次实验围绕 MNIST 手写数字识别任务展开，主要任务如下：

- (1) 完成实验环境配置与基础流程运行。实验在华为云 ModelArts Notebook 环境中进行，使用 MindSpore 深度学习框架，按照指导文档完成代码运行、模型训练和结果记录。
- (2) 完成 MNIST 数据集处理与 LeNet5 模型训练。实验需要下载并读取 MNIST 数据集，对图像数据进行必要预处理，并搭建 LeNet5 卷积神经网络，完成训练集训练和测试集评估。
- (3) 记录并分析模型实验结果。训练过程中需要记录每轮训练后的 Accuracy 和 Avg loss，并对测试样本进行预测可视化，观察模型对手写数字的识别效果。
- (4) 完成拓展实验与对比分析。除基础实验外，课程要求还需要进行拓展尝试，例如更换数据集或调整神经网络结构。结合本次实验内容，本报告选择的拓展方向为改变 LeNet5 网络结构，即在完成基础 LeNet5 训练后，重新设计改进 LeNet5 模型，并比较两种模型在 MNIST 测试集上的准确率和平均损失。

3 实验原理

3.1 MNIST 数据集与预处理

MNIST 是常用于手写数字识别任务的图像分类数据集，包含 0 到 9 共 10 类数字图像。数据集中的图像为单通道灰度图，训练集包含 60000 张图片，测试集包含 10000

张图片。本次实验中，原始图像经过预处理后输入 LeNet5 网络，用于完成 10 分类任务。

原始 MNIST 图像需要经过预处理后才能输入卷积神经网络。本实验中的预处理主要包括图像尺寸调整、像素值缩放、归一化、通道格式转换和批处理。首先，图像尺寸由原始大小调整为 32×32 ，以适配 LeNet5 中卷积层和池化层的结构。其次，像素值由原来的整数范围缩放到 $[0, 1]$ 区间，并使用均值和标准差进行标准化处理。归一化过程可以表示为：

$$x' = \frac{x - \mu}{\sigma}$$

其中， x 表示缩放后的像素值， μ 和 σ 分别表示数据集对应的均值和标准差。此外，MindSpore 卷积层通常采用通道在前的输入格式，因此需要将图像张量由 HWC 格式转换为 CHW 格式。最后，数据集按 batch 组织，使模型能够以小批量方式进行训练和测试。

3.2 卷积神经网络与 LeNet5 结构

卷积神经网络（Convolutional Neural Network, CNN）是一类适合图像处理任务的深度学习模型。与普通全连接网络相比，卷积神经网络通过卷积核在图像局部区域上滑动，能够提取边缘、笔画、角点和局部纹理等空间特征。卷积层的基本计算过程可以表示为：

$$h_{i,j}^{(k)} = \sigma \left(\sum_c \sum_{u,v} w_{u,v,c}^{(k)} x_{i+u,j+v,c} + b^{(k)} \right)$$

其中， x 为输入特征图， w 为卷积核参数， b 为偏置项， σ 为激活函数， $h^{(k)}$ 表示第 k 个输出通道的特征图。池化层用于降低特征图尺寸，减少后续计算量，并增强模型对局部位置变化的鲁棒性。本实验中使用最大池化层，在局部区域内取最大响应值，以保留较显著的特征。

LeNet5 是经典的卷积神经网络结构，常用于手写数字识别任务。本实验中的基础 LeNet5 由两个卷积层、两个池化层和三个全连接层组成。基础 LeNet5 的主要结构为：第一层卷积输出 6 个通道，第二层卷积输出 16 个通道；展平后依次进入 120、84 和 10 维全连接层。最后一层输出不直接表示概率，而是表示每个类别对应的原始预测分数，后续损失函数会基于这些 logits 与真实标签计算模型误差。

3.3 损失函数与评价指标

本实验采用交叉熵损失函数衡量模型预测结果与真实标签之间的差异。对于多分类任务，模型输出 logits 后，可以通过 Softmax 转换为类别概率：

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

其中, z_i 为模型对第 i 类输出的 logits, p_i 表示该类别对应的预测概率。交叉熵损失可以表示为:

$$L = -\frac{1}{N} \sum_{n=1}^N \log p_{n,y_n}$$

其中, N 为 batch 中的样本数量, y_n 为第 n 个样本的真实类别。损失值越小, 说明模型对真实类别的预测概率越高。

模型训练过程中, 优化器根据损失函数对网络参数的梯度更新模型参数。本实验使用 Momentum 优化器, 该方法在梯度下降的基础上引入历史更新方向, 有助于减小训练过程中的震荡, 使参数更新更加稳定。评价指标采用 Accuracy 和 Avg loss。Accuracy 表示测试集中预测正确样本数占总样本数的比例:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}}$$

Avg loss 表示测试集上的平均损失, 用于反映模型整体预测误差。两者结合可以更全面地评价模型性能。

4 实验内容

4.1 实验平台与运行环境

本次实验在华为云 ModelArts Notebook 环境中完成, 主要使用 MindSpore、Python、NumPy 和 Matplotlib 等工具。实验采用在线 Notebook 方式运行, 通过 JupyterLab 打开和编辑代码文件, 并在云端环境中完成数据下载、模型训练和结果保存。

在环境配置过程中, 首先进入华为云 ModelArts 控制台, 并在开发空间中创建 Notebook 实例。创建时选择公共镜像中的 MindSpore 预置环境, 计算资源类型采用 Ascend 相关配置。Notebook 启动后, 在 JupyterLab 页面中运行实验代码。实验相关文件保存在工作目录中, 主要包括实验 notebook、模型参数文件和结果图像。



```

pip install download
Looking in indexes: http://pip.modelarts.private.com:8888/repository/pypi/simple
Collecting download
  Downloading http://pip.modelarts.private.com:8888/repository/pypi/packages/download/0.3.5/download-0.3.5-py3-none-any.whl (8.8 kB)
Requirement already satisfied: tqdm in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from download) (4.66.1)
Requirement already satisfied: requests in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from download) (2.27.1)
Requirement already satisfied: six in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from download) (1.16.0)
Requirement already satisfied: certifi>=2017.4.17 in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from requests->download) (2023.11.17)
Requirement already satisfied: idna<4,>=2.5 in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from requests->download) (2.10)
Requirement already satisfied: charset-normalizer~=2.0.0 in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from requests->download) (2.0.12)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from requests->download) (1.26.7)
Installing collected packages: download
Successfully installed download-0.3.5
WARNING: You are using pip version 21.0.1; however, version 26.0.1 is available.
You should consider upgrading via the '/home/ma-user/anaconda3/envs/MindSpore/bin/python -m pip install --upgrade pip' command.
Note: you may need to restart the kernel to use updated packages.

69 ms [2] %env no_proxy='a.test.com,127.0.0.1,2.2.2.2'
env: no_proxy='a.test.com,127.0.0.1,2.2.2.2'

```

Figure 1: ModelArts Notebook 实验入口与运行环境

实验完成后，及时停止或删除 Notebook 实例，避免云端资源持续占用。

MNIST 数据集包含 0 到 9 共 10 类手写数字图像，每张图片均为灰度图。实验中使用 `download` 工具从 MindSpore 示例数据地址下载数据压缩包，解压后通过 `MnistDataset` 接口读取训练集和测试集。

```
[*] pip install download
Looking in indexes: http://pip.modelarts.private.com:8888/repository/pypi/simple
Collecting download
  Downloading http://pip.modelarts.private.com:8888/repository/pypi/packages/download/0.3.5/download-0.3.5-py3-none-any.whl (8.8 kB)
Requirement already satisfied: tqdm in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from download) (4.66.1)
Requirement already satisfied: requests in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from download) (2.27.1)
Requirement already satisfied: six in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from download) (1.16.0)
Requirement already satisfied: certifi>=2017.4.17 in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from requests->download) (2023.11.17)
Requirement already satisfied: idna<4,>=2.5 in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from requests->download) (2.10)
Requirement already satisfied: charset-normalizer~=2.0.0 in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from requests->download) (2.0.12)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from requests->download) (1.26.7)
Installing collected packages: download
Successfully installed download-0.3.5
WARNING: You are using pip version 21.0.1; however, version 26.0.1 is available.
You should consider upgrading via the '/home/ma-user/anaconda3/envs/MindSpore/bin/python -m pip install --upgrade pip' command.
Note: you may need to restart the kernel to use updated packages.
```

```
69 ms [2] %env no_proxy='a.test.com,127.0.0.1,2.2.2.2'
env: no_proxy='a.test.com,127.0.0.1,2.2.2.2'
```

```
2.7 s from download import download

# 下载MNIST数据集
url = "https://mindspore-website.obs.cn-north-4.myhuaweicloud.com/" \
      "notebook/datasets/MNIST_Data.zip"
path = download(url, ".", kind="zip", replace=True)

Downloading data from https://mindspore-website.obs.cn-north-4.myhuaweicloud.com/notebook/datasets/MNIST_Data.zip (10.3 MB)

file_sizes: 100% ██████████ 10.8M/10.8M [00:00<00:00, 25.3MB/s]
Extracting zip file...
Successfully downloaded / unzipped to ./
```

数据集下载完成后，训练集和测试集分别存放在对应目录中。后续训练和评估均基于这两个数据子集进行，其中训练集用于更新模型参数，测试集用于评价模型在未参与训练样本上的识别效果。

数据处理部分主要由 `datapipe` 函数完成。该函数将数据读取、图像变换、标签类型转换和批处理组织在同一流程中，使训练集和测试集能够以统一格式输入模型。

4

CHW，以符合 MindSpore 卷积层的输入要求。标签部分转换为 int32 类型，最后按照 batch 组织数据，每批包含 32 个样本。

```
131 ms # 定义预处理操作的流程，整体对数据集进行处理
def datapipe(path, batch_size=32):
    image_transform = [
        vision.Resize(size=32, interpolation=Inter.LINEAR), # Resize: 以双线性插值方式调整图像尺寸大小
        vision.Rescale(1.0 / 255.0, 0.0), # Rescale: 缩放图像的像素值大小，将像素值统一除255，数据类型由unit8转为float32
        vision.Normalize(mean=(0.1307,), std=(0.3081,)), # Normalize: 将像素值归一化
        vision.HWC2CHW(), # HWC2CHW: 将张量格式从 (height,width,channel) 转换成(channel,height,width)
    ]
    label_transform = transforms.TypeCast(ms.int32)

    # 利用MnistDataset接口读取解压后的MNIST的训练集和测试集，并进行shuffle操作
    dataset = MnistDataset(path, shuffle=True)
    # 通过map方法对每张图片应用数据处理操作
    dataset = dataset.map(operations=image_transform, input_columns=["image"])
    # 将每个标签的数据类型转换为int32
    dataset = dataset.map(operations=label_transform, input_columns=["label"])
    # 对数据集进行分批处理，当最后一个批处理数据包含的数据条目小于batch_size时，drop_remainder表示是否将该批处理丢弃，不传递给下一个操作，默认值: False, 不
    dataset = dataset.batch(batch_size, drop_remainder=True)
    return dataset

dataset_train = datapipe("MNIST_Data/train")
dataset_eval = datapipe("MNIST_Data/test")
```

Figure 3: 图像尺寸、数值范围和通道顺序的统一处理

为检查数据读取和预处理是否正确，实验中读取了一个 batch 的样本，并输出其张量形状和标签信息。从结果可以看到，图像数据已经转换为批量输入格式，能够直接用于后续模型训练。

```
776 ms [8] data_next = dataset_train.create_dict_iterator(output_numpy=True).__next__()
print('Batch Size/通道数/图像长/宽: ', data_next['image'].shape)
print('图像的标签样式: ', data_next['label'])

plt.figure()
plt.imshow(data_next['image'][1,...].squeeze(), cmap="gray")
plt.grid(False)
plt.show()

Batch Size/通道数/图像长/宽: (32, 1, 32, 32)
图像的标签样式: [4 3 7 5 0 7 7 9 5 3 0 3 6 5 8 6 9 8 5 0 6 4 9 8 7 1 5 4 3 9 0 9]
```

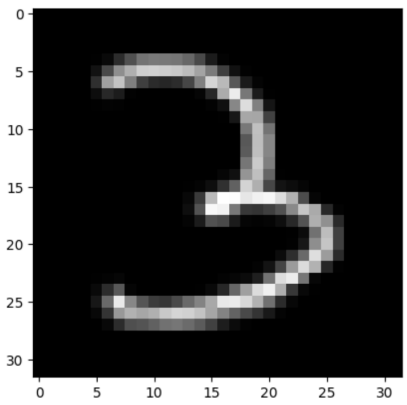


Figure 4: 抽取一个数据结果显示

4.4 基础 LeNet5 模型实现

基础模型采用 LeNet5 卷积神经网络结构。该模型通过前端卷积层提取手写数字图像中的局部笔画和轮廓特征，再通过全连接层完成类别判断。具体结构为：第一层卷积

输出 6 个通道，第二层卷积输出 16 个通道；经过池化和展平后，特征依次输入 120、84 和 10 维全连接层，最后得到 10 个数字类别对应的 logits。

```
73 ms ④ #定义模型结构，MindSpore中的模型时通过construct定义模型结构，在__init__中初始化各层的对象
class LeNet5(nn.Cell):
    """LeNet5"""

    def __init__(self, num_classes=10, num_channel=1):
        super(LeNet5, self).__init__()
        # 卷积层，输入的通道数为num_channel，输出的通道数为6，卷积核大小为5*5
        self.conv1 = nn.Conv2d(num_channel, 6, 5, pad_mode='valid')
        # 卷积层，输入的通道数为6，输出的通道数为16，卷积核大小为5*5
        self.conv2 = nn.Conv2d(6, 16, 5, pad_mode='valid')
        # ReLU激活函数
        self.relu = nn.ReLU()
        # 池化层
        self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
        # 多维数组展平为一维数组
        self.flatten = nn.Flatten()
        # 全连接层，输入个数为16*5*5，输出个数为120
        self.fc1 = nn.Dense(16 * 5 * 5, 120, weight_init=Normal(0.02))
        # 全连接层，输入个数为120，输出个数为84
        self.fc2 = nn.Dense(120, 84, weight_init=Normal(0.02))
        # 全连接层，输入个数为84，分类的个数为num_class
        self.fc3 = nn.Dense(84, num_classes, weight_init=Normal(0.02))

    def construct(self, x):
        # 使用定义好的运算构建前向网络
        x = self.conv1(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Figure 5: 基础 LeNet5 网络的 MindSpore 实现

在 MindSpore 中，神经网络通过继承 `nn.Cell` 定义。网络各层在 `__init__` 方法中完成初始化，前向传播过程则在 `construct` 方法中定义。这样可以使模型结构更加清晰，也便于后续对部分网络层进行修改。

4.5 改进 LeNet5 结构设计

改进模型保留 LeNet5 的基本框架，主要对网络宽度进行调整。考虑到 MNIST 中不同数字在笔画粗细、弯折位置和局部轮廓上存在差异，增加卷积层输出通道数可以使模型学习更多类型的局部特征。因此，改进模型将第一层卷积输出通道由 6 增加到 8，将第二层卷积输出通道由 16 增加到 32。

由于第二层卷积输出通道数发生变化，池化后的展平维度也相应调整为 $32 \times 5 \times 5$ 。在全连接层部分，原始模型中的 120 和 84 维结构被调整为 128 和 64 维。整体来看，改进模型仍保持两层卷积和三层全连接的主体结构，只改变部分层的宽度，以便在相同训练流程下观察结构调整带来的影响。

Table 1: 基础模型与改进模型的结构差异

结构位置	基础 LeNet5	改进 LeNet5
Conv1 输出通道	6	8
Conv2 输出通道	16	32
FC1 维度	120	128
FC2 维度	84	64
输出类别数	10	10

4.6 模型训练与测试方法

训练过程使用自定义的 `train_loop` 完成。每个 batch 的图像数据输入网络后，模型输出 logits，并通过交叉熵损失函数计算预测结果与真实标签之间的误差。随后，使用 `value_and_grad` 进行自动求导，得到损失函数关于模型参数的梯度，再由 Momentum 优化器更新参数。

测试阶段不进行参数更新，只遍历测试集并统计预测正确的样本数量，同时计算测试集上的平均损失。通过在每一轮训练后进行测试，可以观察模型准确率和平均损失的变化情况，从而判断训练过程是否有效。

```

# 对LeNet5进行实例化
network = LeNet5()

# 定义损失函数
net_loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
# 定义优化器：通过model.trainable_params()方法获得模型的可训练参数，并传入学习率超参数初始化优化器
net_opt = nn.Momentum(network.trainable_params(), learning_rate=0.01, momentum=0.9)

# 定义用于训练的train_loop函数。
def train_loop(model, dataset, loss_fn, optimizer):
    # 定义正向计算函数
    def forward_fn(data, label):
        logits = model(data)
        loss = loss_fn(logits, label)
        return loss

    # 定义微分函数，使用mindspore.value_and_grad获得微分函数grad_fn，输出loss和梯度。
    # 由于是对模型参数求导，grad_position 配置为None，传入可训练参数。
    grad_fn = ms.value_and_grad(forward_fn, None, optimizer.parameters)

    # 定义 one-step training 函数
    def train_step(data, label):
        loss, grads = grad_fn(data, label)
        optimizer(grads)
        return loss

    size = dataset.get_dataset_size()
    model.set_train()
    for batch, (data, label) in enumerate(dataset.create_tuple_iterator()):
        loss = train_step(data, label)

        if batch % 100 == 0:
            loss, current = loss.asnumpy(), batch
            print(f"loss: {loss:>7f} [{current:>3d}/{size:>3d}]")

# 定义用于测试的test_loop函数。

```

Figure 6: 训练循环、损失函数与优化器配置

4.7 预测结果可视化

完成训练后，实验从测试集中取出一批样本，加载保存后的模型参数，并使用 `Model.predict` 得到预测类别。预测可视化可以直接展示模型对具体手写数字图像的分类结果，是对 Accuracy 和 Avg loss 等数值指标的补充。

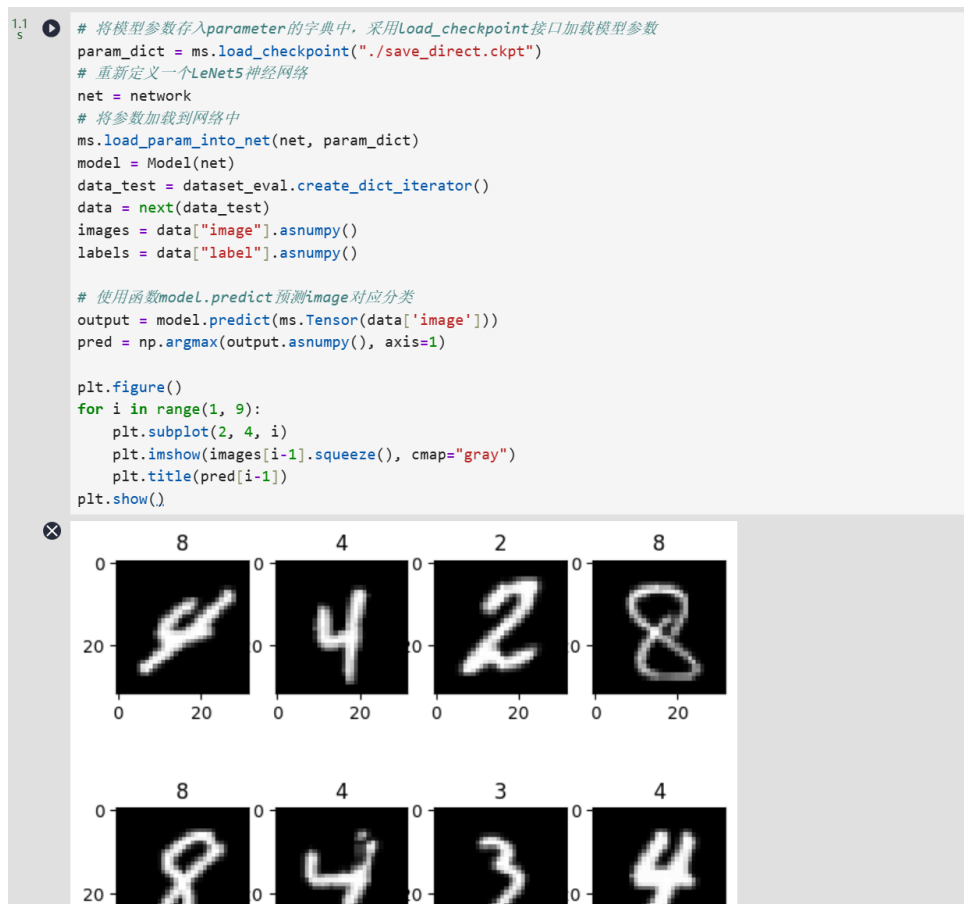


Figure 7: 测试样本的预测类别可视化

5 实验结果

5.1 基础模型训练表现

根据 `src/LeNet-mnist-better.ipynb` 中保留的运行输出，基础 LeNet5 共训练 3 轮。模型在第一轮训练后测试准确率达到 96.4%，第三轮提升到 98.6%。同时，平均损失从 0.113729 降低到 0.043426，说明随着训练进行，模型在测试集上的分类误差逐步下降。

Table 2: 基础 LeNet5 在测试集上的表现

Epoch	Accuracy(%)	Avg loss
1	96.4	0.113729
2	98.4	0.047596
3	98.6	0.043426

```

loss: 2.291700 [ 0/1875]
loss: 2.301501 [100/1875]
loss: 0.851501 [200/1875]
loss: 0.247642 [300/1875]
loss: 0.185502 [400/1875]
loss: 0.039571 [500/1875]
loss: 0.123473 [600/1875]
loss: 0.060880 [700/1875]
loss: 0.063774 [800/1875]
loss: 0.161021 [900/1875]
loss: 0.016864 [1000/1875]
loss: 0.014784 [1100/1875]
loss: 0.063362 [1200/1875]
loss: 0.047109 [1300/1875]
loss: 0.061593 [1400/1875]
loss: 0.053372 [1500/1875]
loss: 0.018991 [1600/1875]
loss: 0.182284 [1700/1875]
loss: 0.000460 [1800/1875]
[ERROR] CORE(8929,ffff834280b0,python
path[/tmp/ipykernel_8929/3728934510.p
[ERROR] CORE(8929,ffff834280b0,python
path[/tmp/ipykernel_8929/3728934510.p
[WARNING] DEVICE(8929,ffff834280b0,py
c:1258) RunInitGraph] Can not find in
/
[WARNING] DEVICE(8929,ffff834280b0,py
c:1258) RunInitGraph] Can not find in
Test:
Accuracy: 96.4%, Avg loss: 0.113729

```

(a) 第一轮测试结果

```

loss: 0.007032 [ 0/1875]
loss: 0.008522 [100/1875]
loss: 0.003023 [200/1875]
loss: 0.002385 [300/1875]
loss: 0.007152 [400/1875]
loss: 0.013098 [500/1875]
loss: 0.178763 [600/1875]
loss: 0.022115 [700/1875]
loss: 0.029586 [800/1875]
loss: 0.013106 [900/1875]
loss: 0.064154 [1000/1875]
loss: 0.007574 [1100/1875]
loss: 0.089548 [1200/1875]
loss: 0.087809 [1300/1875]
loss: 0.055872 [1400/1875]
loss: 0.050827 [1500/1875]
loss: 0.069838 [1600/1875]
loss: 0.135009 [1700/1875]
loss: 0.149320 [1800/1875]
Test:
Accuracy: 98.4%, Avg loss: 0.047596

```

(b) 第二轮测试结果

```

loss: 0.002814 [ 0/1875]
loss: 0.156942 [100/1875]
loss: 0.035236 [200/1875]
loss: 0.002097 [300/1875]
loss: 0.114576 [400/1875]
loss: 0.004368 [500/1875]
loss: 0.311557 [600/1875]
loss: 0.002213 [700/1875]
loss: 0.001277 [800/1875]
loss: 0.002771 [900/1875]
loss: 0.058117 [1000/1875]
loss: 0.161126 [1100/1875]
loss: 0.015304 [1200/1875]
loss: 0.019098 [1300/1875]
loss: 0.003421 [1400/1875]
loss: 0.006040 [1500/1875]
loss: 0.040492 [1600/1875]
loss: 0.000816 [1700/1875]
loss: 0.001467 [1800/1875]
Test:
Accuracy: 98.6%, Avg loss: 0.043426
Done!

```

(c) 第三轮测试结果

Figure 8: 基础 LeNet5 三轮训练后的测试结果

5.2 改进模型训练表现

根据 `src/LeNet-mnist-better.ipynb` 中改进模型单元的输出，改进 LeNet5 同样训练 3 轮。测试集准确率由第一轮的 98.1% 提升到第三轮的 98.8%，平均损失由 0.057485 降低到 0.037108。与基础模型相比，改进模型在第三轮取得了略高的准确率和更低的平均损失。

Table 3: 改进 LeNet5 训练过程中的性能变化

Epoch	Accuracy(%)	Avg loss
1	98.1	0.057485
2	98.4	0.048206
3	98.8	0.037108

```

loss: 2.308430 [ 0/1875]
loss: 2.272609 [100/1875]
loss: 1.235499 [200/1875]
loss: 1.246167 [300/1875]
loss: 0.761702 [400/1875]
loss: 0.124194 [500/1875]
loss: 0.235526 [600/1875]
loss: 0.015912 [700/1875]
loss: 0.091912 [800/1875]
loss: 0.024375 [900/1875]
loss: 0.077978 [1000/1875]
loss: 0.160003 [1100/1875]
loss: 0.010996 [1200/1875]
loss: 0.114518 [1300/1875]
loss: 0.014707 [1400/1875]
loss: 0.029423 [1500/1875]
loss: 0.018419 [1600/1875]
loss: 0.072639 [1700/1875]
loss: 0.006268 [1800/1875]
[ERROR] CORE(8929,ffff834280b0,python:
path[/tmp/ipykernel_8929/4161387676.py]
[ERROR] CORE(8929,ffff834280b0,python:
path[/tmp/ipykernel_8929/4161387676.py]
[WARNING] DEVICE(8929,ffff834280b0,p
c:1258) RunInitGraph] Can not find in:
/
[WARNING] DEVICE(8929,ffff834280b0,p
c:1258) RunInitGraph] Can not find in:
Test:
Accuracy: 98.1%, Avg loss: 0.057485

```

(a) 第一轮测试结果

```

loss: 0.077753 [ 0/1875]
loss: 0.034437 [100/1875]
loss: 0.001838 [200/1875]
loss: 0.003762 [300/1875]
loss: 0.007404 [400/1875]
loss: 0.007600 [500/1875]
loss: 0.008092 [600/1875]
loss: 0.008279 [700/1875]
loss: 0.002720 [800/1875]
loss: 0.012172 [900/1875]
loss: 0.047090 [1000/1875]
loss: 0.292217 [1100/1875]
loss: 0.011565 [1200/1875]
loss: 0.105856 [1300/1875]
loss: 0.013531 [1400/1875]
loss: 0.012130 [1500/1875]
loss: 0.136206 [1600/1875]
loss: 0.118684 [1700/1875]
loss: 0.071440 [1800/1875]
Test:
Accuracy: 98.4%, Avg loss: 0.048206

```

(b) 第二轮测试结果

```

loss: 0.162068 [ 0/1875]
loss: 0.015569 [100/1875]
loss: 0.001325 [200/1875]
loss: 0.006787 [300/1875]
loss: 0.074011 [400/1875]
loss: 0.000104 [500/1875]
loss: 0.001328 [600/1875]
loss: 0.000679 [700/1875]
loss: 0.011765 [800/1875]
loss: 0.009111 [900/1875]
loss: 0.001974 [1000/1875]
loss: 0.051074 [1100/1875]
loss: 0.000386 [1200/1875]
loss: 0.052453 [1300/1875]
loss: 0.078756 [1400/1875]
loss: 0.237101 [1500/1875]
loss: 0.008714 [1600/1875]
loss: 0.005823 [1700/1875]
loss: 0.001683 [1800/1875]
Test:
Accuracy: 98.8%, Avg loss: 0.037108

```

(c) 第三轮测试结果

Figure 9: 改进 LeNet5 三轮训练后的测试结果

5.3 最终性能对比

两种模型的最终结果如表 4 所示。这里统一采用 `src/LeNet-mnist-better.ipynb` 中基础模型和改进模型各自第三轮训练后的测试输出作为对比口径。改进模型相比基础模型准确率提高 0.2 个百分点，平均损失降低 0.006318。从准确率看，两种模型差距较小；从平均损失看，改进模型在测试集上的整体误差更低。

Table 4: 两种 LeNet5 结构的最终测试结果对照

模型	最终 Epoch	Accuracy(%)	Avg loss
基础 LeNet5	3	98.6	0.043426
改进 LeNet5	3	98.8	0.037108
差值 (改进-基础)	—	+0.2	-0.006318

为了更直观地比较两种模型的结果，进一步绘制最终测试指标柱状图和三轮训练过程折线图。柱状图同时展示最终 Accuracy 和 Avg loss，折线图展示 Accuracy 与 Avg loss 随 epoch 的变化趋势。

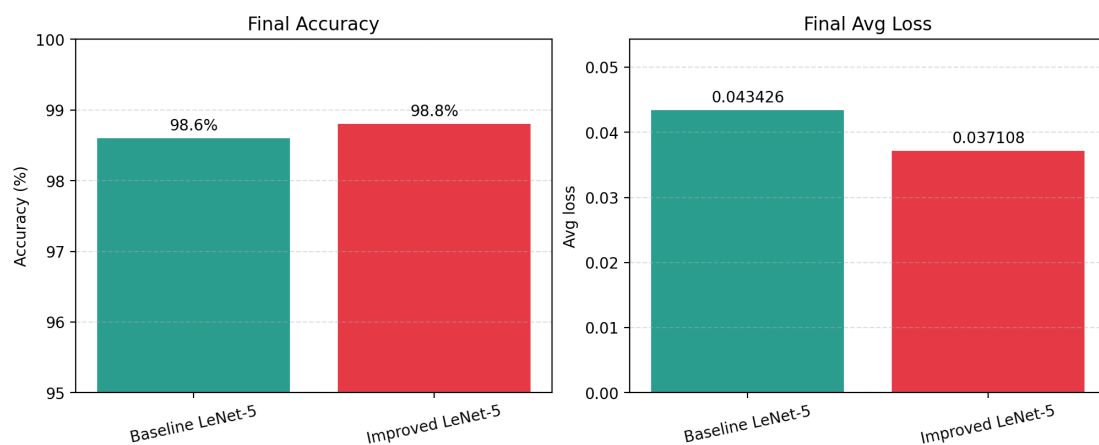


Figure 10: 基础模型与改进模型最终测试指标柱状对比

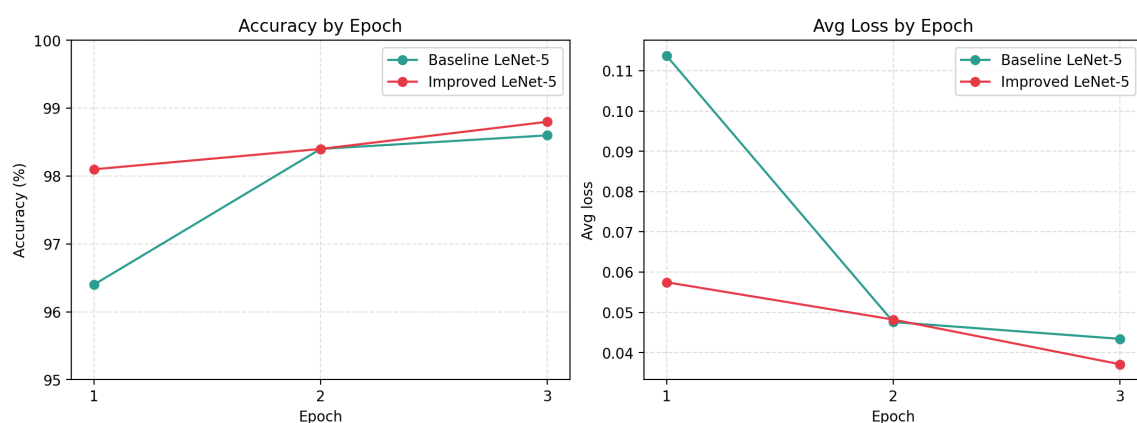


Figure 11: 基础模型与改进模型训练过程指标变化对比

6 结果分析

6.1 训练轮数与收敛状态

从基础模型结果可以看出，LeNet5 在 MNIST 数据集上的收敛速度较快。第一轮训练后，测试准确率已经达到 96.4%；第三轮训练后，准确率进一步提高到 98.6%，平均损失也降低到 0.043426。

这说明基础 LeNet5 已经能够有效学习 MNIST 图像中的笔画和轮廓特征。改进模型第一轮训练后的准确率为 98.1%，高于基础模型第一轮结果。第三轮训练后，改进模型准确率达到 98.8%，相较本次基础模型训练输出提高 0.2 个百分点。该结果表明，增加卷积通道数和调整全连接层规模后，模型在测试集上的识别效果略有提升，但提升幅度并不大。这与 MNIST 数据集任务难度较低、基础模型已经具备较强识别能力有关。

6.2 网络结构变化的影响

改进结构的主要变化在于卷积层通道数增加。通道数增加后，同一层卷积可以学习更多类型的局部特征，例如笔画边缘、局部弯折和数字轮廓组合。因此，改进模型的特征表达能力相较基础模型有所增强。

从最终对比结果看，改进模型最终 Avg loss 为 0.037108，低于基础模型的 0.043426。平均损失下降说明改进模型在测试集上的整体预测误差更低。与此同时，准确率只提高了 0.2 个百分点，说明损失下降并不一定会直接对应到大量新增的正确分类。对于基础模型已经能够正确识别的样本，改进模型可能主要表现为对正确类别输出更高的概率；而对于少数书写模糊或类别边界接近的样本，仅增加网络宽度仍然难以完全避免误判。

因此，本次结构改进的效果主要体现在平均损失下降，而不是准确率的大幅提升。这也说明在模型性能已经较高的情况下，仅调整网络宽度带来的边际收益有限。

6.3 实验不足与改进方向

本实验仍存在一定不足。首先，两种模型均只训练 3 个 epoch，实验结果反映的是较短训练过程中的表现，不能充分说明改进模型在更长训练过程中的稳定性。其次，本次拓展实验只调整了卷积通道数和全连接层规模，没有进一步比较 Dropout、学习率调度、数据增强或正则化等方法的影响。

因此，当前结论更适合说明在相同训练配置下，适度加宽 LeNet5 可以带来有限提升，而不能直接说明该结构为最优模型。后续若继续完善实验，可以增加更多对照组，从而更清楚地分析不同结构变化对结果的影响。也可以增加训练轮数并记录更完整的准确率和损失曲线，以观察模型是否继续收敛，或是否出现过拟合现象。

7 实验总结

本次实验中，我主要完成了基于 MindSpore 和 ModelArts 的 MNIST 手写数字识别流程。实验首先配置 Notebook 运行环境，下载并预处理 MNIST 数据集；随后实现基础 LeNet5 网络，完成模型训练、测试评估和预测可视化；最后在基础模型上设计改进 LeNet5，通过增加卷积通道数和调整全连接层维度进行对比实验。从实验结果来看，基础 LeNet5 在 MNIST 测试集上达到 98.6% 的准确率，说明经典卷积神经网络结构能够较好地完成低分辨率灰度数字图像分类任务。改进模型最终准确率达到 98.8%，平均损失降低到 0.037108，表明网络结构加宽后模型性能有一定改善，但提升幅度有限。

通过本次实验，我可以较完整地理解 MindSpore 中数据集读取、图像预处理、网络定义、自动微分训练、损失函数计算、优化器更新和模型评估等基本流程。同时，实验结果也表明，网络结构调整需要结合数据集难度、训练配置和评价指标综合分析，不能仅以模型复杂度作为判断依据。

A 核心代码

附录中选取的是改进 LeNet-5 的核心代码片段，主要用于说明改进 LeNet5 的实现方式和结果统计方法。代码首先给出改进网络的结构定义，包括卷积通道数和全连接层维度的调整；随后保留训练函数与测试函数，展示模型参数更新、Accuracy 统计和 Avg loss 计算过程；最后列出基础模型与改进模型第三轮测试结果的对比变量。

```
class ImprovedLeNet5(nn.Cell):
    """Improved LeNet-5 for MNIST classification"""

    def __init__(self, num_classes=10, num_channel=1):
        super(ImprovedLeNet5, self).__init__()
        self.conv1 = nn.Conv2d(
            in_channels=num_channel,
            out_channels=8,
            kernel_size=5,
            pad_mode='valid'
        )
        self.conv2 = nn.Conv2d(
            in_channels=8,
            out_channels=32,
            kernel_size=5,
            pad_mode='valid'
        )
        self.relu = nn.ReLU()
        self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Dense(32 * 5 * 5, 128, weight_init=Normal(0.02))
        self.fc2 = nn.Dense(128, 64, weight_init=Normal(0.02))
        self.fc3 = nn.Dense(64, num_classes, weight_init=Normal(0.02))

    def construct(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.flatten(x)
```

```
x = self.relu(self.fc1(x))
x = self.relu(self.fc2(x))
x = self.fc3(x)
return x

def improved_train_loop(model, dataset, loss_fn, optimizer):
    def forward_fn(data, label):
        logits = model(data)
        loss = loss_fn(logits, label)
        return loss

    grad_fn = ms.value_and_grad(forward_fn, None, optimizer.parameters)

    def train_step(data, label):
        loss, grads = grad_fn(data, label)
        optimizer(grads)
        return loss

    size = dataset.get_dataset_size()
    model.set_train()
    for batch, (data, label) in enumerate(dataset.create_tuple_iterator()):
        loss = train_step(data, label)
        if batch % 100 == 0:
            print(f"loss: {loss.asnumpy():>7f} [{batch:>3d}/{size:>3d}]")

def improved_test_loop(model, dataset, loss_fn):
    num_batches = dataset.get_dataset_size()
    model.set_train(False)
    total, test_loss, correct = 0, 0, 0

    for data, label in dataset.create_tuple_iterator():
        pred = model(data)
        total += data.shape[0]
        test_loss += loss_fn(pred, label).asnumpy()
        correct += (pred.argmax(1) == label).asnumpy().sum()
```

```
test_loss /= num_batches
accuracy = correct / total
print(f"Accuracy: {(100 * accuracy):>0.1f}%, "
      f"Avg loss: {test_loss:>8f}")
return accuracy, test_loss

origin_final_accuracy = 98.6
origin_final_loss = 0.043426
improved_final_accuracy = 98.8
improved_final_loss = 0.037108

print("Accuracy improvement:",
      improved_final_accuracy - origin_final_accuracy)
print("Loss change:",
      improved_final_loss - origin_final_loss)
```